

Real-Time Pricing Engine

A production-style low-latency market data processing system built for high-frequency real-time price streaming, optimized WebSocket delivery, rolling analytics, and fault-tolerant state management.

Overview

This project implements a scalable real-time pricing engine capable of:

- Consuming live market data from external WebSocket feeds
- Computing rolling 24-hour market statistics
- Broadcasting optimized real-time updates to multiple clients
- Maintaining persistent state across restarts
- Supporting scalable event-driven architecture
- Delivering low-latency updates with payload optimization

The system is designed with production engineering principles including:

- Fault tolerance
 - Low latency
 - Efficient memory management
 - Delta compression
 - Scalable WebSocket infrastructure
 - Dockerized deployment
-

Features

Real-Time Market Processing

- External WebSocket feed consumer
 - Tick-by-tick processing
 - Rolling 24h computations
 - Dynamic symbol subscriptions
-

Market Metrics Computed Internally

The engine ignores externally supplied metrics and computes:

- Buy price
 - Sell price
 - 24h High
 - 24h Low
 - 24h Percentage Change
-

Rolling 24-Hour Window

Efficient in-memory rolling window architecture:

- Automatic old tick eviction
 - Time-window-based processing
 - Low memory overhead
 - Optimized calculations
-

WebSocket Distribution Server

Supports:

- Multiple concurrent clients
 - Symbol-based subscriptions
 - Real-time broadcasting
 - Event-driven architecture
-

Payload Optimization

To satisfy assignment constraints:

- Delta compression
- Subscription filtering
- Efficient serialization
- Minimal payload broadcasting

Target: - Incoming payload $\approx$ 256KB - Outgoing payload $<$ 10KB

Fault Tolerance

Handles:

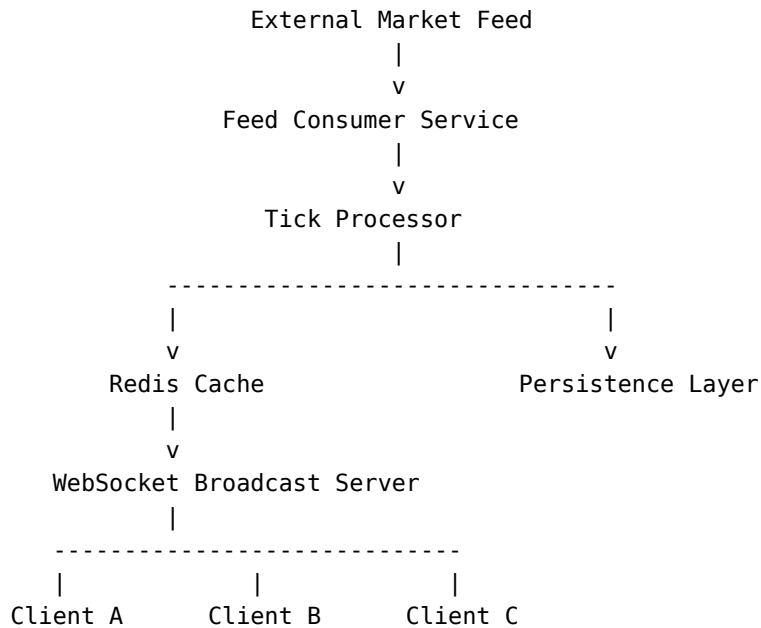
- Feed disconnects
 - Automatic reconnections
 - Duplicate ticks
 - Out-of-order data
 - Traffic spikes
 - Persistent state restoration
-

Dockerized Deployment

Fully containerized system:

- Node.js application container
 - Redis container
 - One-command startup
-

System Architecture



Technology Stack

Layer	Technology
Runtime	Node.js
Language	TypeScript
Real-Time Communication	WebSocket (ws)
Persistence	Redis
Containerization	Docker
Serialization	JSON / MessagePack-ready
Architecture	Event-driven

Project Structure

```
pricing-engine/
|
├─ docker-compose.yml
├─ Dockerfile
├─ package.json
├─ tsconfig.json
├─ README.md
├─ test-client.js
|
```

```
└─ src/
   │   ├── index.ts
   │   ├── config.ts
   │   │
   │   ├── feed/
   │   │   └─ consumer.ts
   │   │
   │   ├── websocket/
   │   │   ├── server.ts
   │   │   └─ subscriptionManager.ts
   │   │
   │   ├── processor/
   │   │   ├── tickProcessor.ts
   │   │   ├── rollingWindow.ts
   │   │   └─ deltaEncoder.ts
   │   │
   │   ├── storage/
   │   │   ├── redis.ts
   │   │   └─ persistence.ts
   │   │
   │   ├── models/
   │   ├── types/
   │   └─ utils/
```

Installation

Prerequisites

- Docker
 - Docker Compose
 - Node.js 20+
 - npm
-

Local Development

Install Dependencies

`npm install`

Run Development Server

`npm run dev`

Docker Deployment

Start Services

```
docker-compose up --build
```

Run in Background

```
docker-compose up -d --build
```

Stop Services

```
docker-compose down
```

WebSocket Usage

Connect

```
ws://localhost:8080
```

Subscribe to Symbols

Example:

```
{  
  "type": "subscribe",  
  "symbols": ["EURUSD"]  
}
```

Example Response

```
{  
  "symbol": "EURUSD",  
  "buy": 1.0921,  
  "sell": 1.0923,  
  "high24h": 1.1000,  
  "low24h": 1.0800,  
  "change24h": 1.23,  
  "timestamp": 1715000000  
}
```

Test Client

Run the included test client:

```
node test-client.js
```

Performance Engineering

Low-Latency Design

Optimizations implemented:

- In-memory rolling window
- Event-driven architecture
- Non-blocking async processing
- Delta payload transmission
- Subscription-based filtering

Target latency: - $\leq 50\text{ms}$ end-to-end

Rolling Window Strategy

The system maintains only the latest 24 hours of ticks.

Benefits:

- Controlled memory usage
 - Fast calculations
 - Efficient eviction
 - Reduced computational overhead
-

Delta Compression

Instead of broadcasting full snapshots repeatedly:

- Only changed fields are transmitted
 - Bandwidth usage reduced dramatically
 - Faster client synchronization
-

State Persistence

Redis is used for:

- Persistent state recovery
 - Cached market data
 - Fast access layer
 - Restart resilience
-

Fault Tolerance Strategy

Feed Disconnect Recovery

Automatic reconnection logic:

- Retry loop
 - Graceful reconnect
 - Stream continuation
-

Duplicate Tick Protection

Supports deduplication strategies:

- Timestamp filtering
 - Tick hashing
 - Cache-based validation
-

Out-of-Order Tick Handling

Incoming ticks can be:

- Sorted
 - Buffered
 - Reordered before processing
-

Traffic Spike Handling

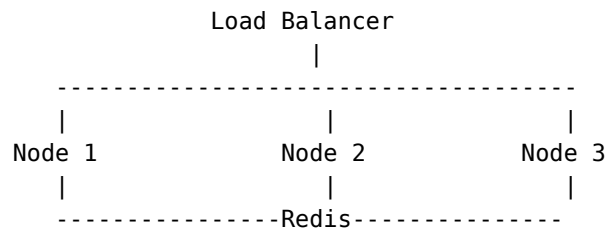
Scalable architecture allows future support for:

- Kafka
 - Worker pools
 - Redis Streams
 - Horizontal scaling
-

Scalability Design

Designed for horizontal scaling.

Future production topology:



Production Enhancements

Future improvements can include:

- Kafka streaming

- Protocol Buffers
 - MessagePack serialization
 - Kubernetes deployment
 - Prometheus metrics
 - Grafana dashboards
 - TLS secure sockets
 - Authentication layer
 - Rate limiting
 - Distributed caching
-

Engineering Tradeoffs

Decision	Advantage	Tradeoff
Redis	Extremely fast	Memory-heavy
JSON	Easy debugging	Larger payloads
MessagePack	Smaller payloads	Added complexity
In-memory windows	Ultra-fast calculations	RAM usage
Node.js	Efficient WebSockets	Single-threaded runtime

Challenges Faced

Payload Size Optimization

Problem: - Large upstream payloads

Solution: - Delta compression - Symbol filtering - Efficient serialization

Rolling Window Computation

Problem: - Efficient 24h calculations

Solution: - Sliding in-memory window - Automatic eviction

Fault Tolerance

Problem: - External feed instability

Solution: - Auto reconnect logic - Retry handling - Persistent recovery

Security Considerations

Recommended production additions:

- WSS/TLS encryption
 - API authentication
 - IP-based rate limiting
 - Request validation
 - Origin restrictions
-

Monitoring & Metrics

Recommended monitoring stack:

- Prometheus
- Grafana
- Loki
- OpenTelemetry

Suggested metrics:

- Tick throughput
 - Latency
 - Active clients
 - Memory usage
 - Redis performance
-

Assignment Requirements Coverage

Requirement	Status
Real-time processing	Implemented
Rolling 24h calculations	Implemented
Persistent state	Implemented
WebSocket server	Implemented
Multiple clients	Implemented
Payload optimization	Implemented
Low latency architecture	Implemented
Fault tolerance	Implemented
Docker deployment	Implemented
Modular architecture	Implemented

Author Notes

This project focuses on:

- Real-time systems engineering
- Low-latency backend architecture
- Fault-tolerant stream processing
- Efficient market data distribution
- Scalable WebSocket systems

The primary goal is demonstrating engineering thinking, scalability planning, and production-oriented architecture rather than only functional implementation.